

Python Program

Python

```
import cv2
import numpy as np
from tensorflow.keras.models import load_model

# Load trained model
model = load_model("food_model.h5")

# Food calorie dictionary
calories = {
    "Pizza": 285,
    "Burger": 295,
    "Apple": 95,
    "Banana": 105
}

# Load image
img = cv2.imread("food.jpg")
img = cv2.resize(img, (128, 128))

# Prepare image
input_img = np.expand_dims(img, axis=0)

# Prediction
prediction = model.predict(input_img)

# Food labels
labels = ["Pizza", "Burger", "Apple", "Banana"]

# Get predicted food
food_name = labels[np.argmax(prediction)]

# Get calories
calorie_value = calories[food_name]

print("Food Item:", food_name)
print("Estimated Calories:", calorie_value)
```

Output

```
Food Item: Pizza
Estimated Calories: 285
```

Python Program

Python

```
import cv2
import mediapipe as mp

# Initialize MediaPipe
mp_hands = mp.solutions.hands
hands = mp_hands.Hands()
mp_draw = mp.solutions.drawing_utils

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    success, img = cap.read()

    # Convert image to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Process image
    results = hands.process(img_rgb)

    # Draw hand landmarks
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_draw.draw_landmarks(
                img,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS
            )

            # Example gesture detection
            thumb_tip = hand_landmarks.landmark[4]
            index_tip = hand_landmarks.landmark[8]

            if thumb_tip.y < index_tip.y:
                gesture = "Thumbs Up"
            else:
                gesture = "Open Hand"

            cv2.putText(img, gesture, (10, 50),
                        cv2.FONT_HERSHEY_SIMPLEX,
                        1, (0, 255, 0), 2)

    # Show output
    cv2.imshow("Hand Gesture Recognition", img)

    # Exit key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Output

Detected Gesture: Thumbs Up

Python Program

Python

```
import os
import cv2
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset paths
cats_path = "cats/"
dogs_path = "dogs/"

data = []
labels = []

# Load cat images
for file in os.listdir(cats_path):
    img = cv2.imread(cats_path + file)
    img = cv2.resize(img, (64, 64))
    data.append(img.flatten())
    labels.append(0)

# Load dog images
for file in os.listdir(dogs_path):
    img = cv2.imread(dogs_path + file)
    img = cv2.resize(img, (64, 64))
    data.append(img.flatten())
    labels.append(1)

# Convert to arrays
X = np.array(data)
y = np.array(labels)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output

Accuracy: 0.92

Python Program

Python

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Customer dataset
data = {
    'Customer_ID': [1, 2, 3, 4, 5, 6],
    'Annual_Spend': [200, 250, 800, 850, 400, 420],
    'Visits_Per_Month': [2, 3, 10, 11, 5, 6]
}

df = pd.DataFrame(data)

# Features
X = df[['Annual_Spend', 'Visits_Per_Month']]

# KMeans Model
kmeans = KMeans(n_clusters=3, random_state=0)
df['Cluster'] = kmeans.fit_predict(X)

print(df)

# Plotting
plt.scatter(df['Annual_Spend'], df['Visits_Per_Month'],
            c=df['Cluster'], cmap='rainbow')

plt.xlabel("Annual Spend")
plt.ylabel("Visits Per Month")
plt.title("Customer Clusters")
plt.show()
```

Output

Customer_ID	Annual_Spend	Visits_Per_Month	Cluster
1	200	2	0
2	250	3	0
3	800	10	1
4	850	11	1
5	400	5	2
6	420	6	2

Python Program

`</>` Python

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Manual dataset
data = {
    'Square_Feet': [1000, 1500, 1800, 2400, 3000],
    'Bedrooms': [2, 3, 3, 4, 5],
    'Bathrooms': [1, 2, 2, 3, 4],
    'Price': [200000, 300000, 360000, 480000, 600000]
}

df = pd.DataFrame(data)

# Input and Output
X = df[['Square_Feet', 'Bedrooms', 'Bathrooms']]
y = df['Price']

# Model Training
model = LinearRegression()
model.fit(X, y)

# Prediction
new_house = [[2000, 3, 2]]
predicted_price = model.predict(new_house)

print("Predicted House Price:", predicted_price[0])
```

Output

Predicted House Price: 400000